

EC200A-CN(TA) QuecOpen UART 开发指导

LTE Standard 模块系列

版本：1.0.0

日期：2022-08-09

状态：临时文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登陆网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2022，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2022.

文档历史

修订记录

版本	日期	作者	变更表述
-	2021-11-25	Larry ZHANG	文档创建
1.0.0	2022-08-09	Jayden CHEN	临时版本

目录

文档历史	3
目录	4
表格索引	5
1 引言	6
2 UART 功能和引脚综述.....	7
2.1. 功能综述	7
2.2. 引脚定义	7
3 UART 设备树配置	9
3.1. 配置说明	9
3.2. DEBUG_UART 设备树配置.....	9
3.3. MAIN_UART 设备树配置	10
4 UART 编译示例及功能验证	12
4.1. 编译步骤	12
4.2. 功能验证	12
4.2.1. 未使能流控	13
4.2.2. 使能硬件流控	13
5 附录 参考文档及术语缩写	15

表格索引

表 1: UART 接口功能综述	7
表 2: DEBUG_UART (设备树节点: UART1) 引脚定义	7
表 3: MAIN_UART (设备树节点: UART2) 引脚定义	8
表 4: UART 与设备树文件中字段的对应关系	9
表 5: 参考文档	15
表 6: 术语缩写	15

1 引言

移远通信 EC200A-CN(TA)模块支持 QuecOpen®方案。QuecOpen®是基于 Linux 的嵌入式开发平台，可简化应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

本文档主要介绍 QuecOpen®方案下，EC200A-CN(TA)模块的 UART 功能开发，包含 UART 接口引脚定义、设备树配置、功能验证和驱动调试的方法等。

Preliminary Document
Not Checked

2 UART 功能和引脚综述

2.1. 功能综述

QuecOpen®方案下，EC200A-CN(TA)模块支持 2 路 UART 接口：

- DEBUG_UART
- MAIN_UART

表 1: UART 接口功能综述

UART 类型	支持的波特率（bps）	默认波特率（bps）	功能描述
DEBUG_UART	115200	115200	● 用于 Linux 控制和日志输出 ● 不支持 RTS/CTS 硬件流控
MAIN_UART	4800、9600、19200、 38400、57600、115200、 230400、460800、921600	115200	● 普通外设通信 UART ● 支持 RTS/CTS 硬件流控

备注

有关串口引脚复用的更多信息，请参考第 2.2 章和文档 [2]。

2.2. 引脚定义

表 2: DEBUG_UART（设备树节点：uart1）引脚定义

引脚名	引脚号	I/O	描述	备注
DBG_RXD	11	DI	UART_RXD	— 仅用于 Linux 控制和日志输出
DBG_TXD	12	DO	UART_TXD	

表 3: MAIN_UART（设备树节点：uart2）引脚定义

引脚名	引脚号	I/O	功能描述	
			复用功能 1（默认）	复用功能 2
MAIN_CTS	64	DO	UART_CTS	GPIO_32
MAIN_RTS	65	DI	UART_RTS	GPIO_31
MAIN_TXD	67	DO	UART_TXD	GPIO_52
MAIN_RXD	68	DI	UART_RXD	GPIO_51

备注

通过复用,GNSS_UART 和 MAIN_UART 接口也可作为 GPIO 功能使用；有关模块 GPIO 引脚配置的详细信息，请参考文档[2]。

3 UART 设备树配置

本章介绍 UART 设备树配置信息。模块各串口在软件设备树文件（*.dtsi* 文件）中对应的字段如下表所示：

表 4：UART 与设备树文件中字段的对应关系

UART 类型	设备树文件中的对应字段
DEBUG_UART	blsp1_uart1
MAIN_UART	blsp1_uart2
GNSS_UART	blsp1_uart3

3.1. 配置说明

模块支持有关 UART 兼容的驱动、引脚选择、寄存器地址、UART 中断号、时钟、系统休眠等功能配置，用户可以根据需要打开或关闭相应功能。

3.2. DEBUG_UART 设备树配置

DEBUG_UART 禁用硬件流控功能，在模块内部显示为设备节点 */dev/ttyS0*；另外，DEBUG_UART 默认用于 Linux 调试和日志输出，不支持复用为其他功能，因此不允许修改该功能配置。

```

root@Openwrt:/# ls /dev/tty*
/dev/tty      /dev/tty25    /dev/tty42    /dev/tty6
/dev/tty0     /dev/tty26    /dev/tty43    /dev/tty60
/dev/tty1     /dev/tty27    /dev/tty44    /dev/tty61
/dev/tty10    /dev/tty28    /dev/tty45    /dev/tty62
/dev/tty11    /dev/tty29    /dev/tty46    /dev/tty63
/dev/tty12    /dev/tty3     /dev/tty47    /dev/tty7
/dev/tty13    /dev/tty30    /dev/tty48    /dev/tty8
/dev/tty14    /dev/tty31    /dev/tty49    /dev/tty9
/dev/tty15    /dev/tty32    /dev/tty5     /dev/ttyGS0
/dev/tty16    /dev/tty33    /dev/tty50    /dev/ttyGS1
/dev/tty17    /dev/tty34    /dev/tty51    /dev/ttyS0
/dev/tty18    /dev/tty35    /dev/tty52    /dev/ttyS1
/dev/tty19    /dev/tty36    /dev/tty53    /dev/ttyS2
/dev/tty2     /dev/tty37    /dev/tty54    /dev/tty_debug0
/dev/tty20    /dev/tty38    /dev/tty55    /dev/ttydiag0
/dev/tty21    /dev/tty39    /dev/tty56    /dev/ttymodem0
/dev/tty22    /dev/tty4     /dev/tty57
/dev/tty23    /dev/tty40    /dev/tty58
/dev/tty24    /dev/tty41    /dev/tty59
root@Openwrt:/#

```

DEBUG_UART 设备树文件的路径为: `ql-ol-kernel/arch/arm/boot/dts/asr1803-p401.dts`。

```

uart1: uart@d4017000 { /* nezhas evb use ap uart */
    pinctrl-names = "default", "sleep";
    pinctrl-0 = <&uart1_pmx_func1 &uart1_pmx_func2>;
    pinctrl-1 = <&uart1_pmx_func1_sleep &uart1_pmx_func2>;
    edge-wakeup-gpio = <&gpio 29 0>; /* GPIO29: AP UART rx pin */
    status = "okay";
};

```

3.3. MAIN_UART 设备树配置

MAIN_UART 默认支持硬件流控，在模块内部显示为设备节点/dev/ttyS1，如下所示：

```

root@openwrt:/# ls /dev/tty*
/dev/tty      /dev/tty25    /dev/tty42    /dev/tty6
/dev/tty0     /dev/tty26    /dev/tty43    /dev/tty60
/dev/tty1     /dev/tty27    /dev/tty44    /dev/tty61
/dev/tty10    /dev/tty28    /dev/tty45    /dev/tty62
/dev/tty11    /dev/tty29    /dev/tty46    /dev/tty63
/dev/tty12    /dev/tty3     /dev/tty47    /dev/tty7
/dev/tty13    /dev/tty30    /dev/tty48    /dev/tty8
/dev/tty14    /dev/tty31    /dev/tty49    /dev/tty9
/dev/tty15    /dev/tty32    /dev/tty5     /dev/ttyGS0
/dev/tty16    /dev/tty33    /dev/tty50    /dev/ttyGS1
/dev/tty17    /dev/tty34    /dev/tty51    /dev/ttyS0
/dev/tty18    /dev/tty35    /dev/tty52    /dev/ttyS1
/dev/tty19    /dev/tty36    /dev/tty53    /dev/ttyS2
/dev/tty2     /dev/tty37    /dev/tty54    /dev/tty_debug0
/dev/tty20    /dev/tty38    /dev/tty55    /dev/ttydiag0
/dev/tty21    /dev/tty39    /dev/tty56    /dev/ttymodem0
/dev/tty22    /dev/tty4     /dev/tty57
/dev/tty23    /dev/tty40    /dev/tty58
/dev/tty24    /dev/tty41    /dev/tty59
root@openwrt:/#

```

MAIN_UART 配置文件路径为 `ql-ol-kernel/arch/arm/boot/dts/asr1803-p401.dts`。设备树节点 `/dev/ttyS1` 在配置文件中默认存在，即 MAIN_UART 功能默认打开（**status = "okay"**），如下图所示：

```
uart2: uart@4036000 {
    pinctrl-names = "default";
    pinctrl-0 = <&gps_pmx_uart_rxd &gps_pmx_uart_txd>;
    status = "okay";
};
```

一般情况下，推荐将 MAIN_UART 作为默认串口使用，或者修改为 GPIO 使用。如需将 MAIN_UART 用作普通 GPIO，需将 **status = "ok"** 改为 **status = "disabled"** 以关闭 MAIN_UART 的功能（此时设备节点 `/dev/ttyS1` 将不存在）。

完成设备树配置后，可参考第 4.1 章所述步骤完成内核的编译，然后将编译生成的内核镜像文件烧录到模块，即可使配置生效。

4 UART 编译示例及功能验证

本章所述示例均摘自 *sample/uart*，SDK 包中包含了 UART 功能的编译示例。示例演示如何将数据从 TX 发送出来以及如何读取 RX 接收的数据，用户可自行查看完整示例。本章节以 MAIN_UART 为例进行介绍如何进行串口开发及功能验证。

4.1. 编译步骤

步骤一：示例中，UART 发送的数据信息如下：

```
int main(int argc, char* argv[])
{
    int iRet;
    int baudRate;
    char strTmp[] = "uart test, +=_0)9(8*7&6^5%4$3#2@1!`~)\n";

    if (argc < 2)
    {
        printf("Usage: %s <baud rate> \n", argv[0]);
        return -1;
    }
}
```

步骤二：进入目录 *sample/uart*，执行 **make** 命令生成可执行文件 *uart_test*。

```
ol@ol:~/1803_sec/ql-ol-extsdk-ec200acntar02a01m2gv01/sample/uart$ make
/opt/ql_crosstools/ql-ec200a-1803e-gcc-8.4.0-v1-toolchain/bin/arm-openwrt-linux-gcc -o main.o -c main.c -march=armv7-a
-marm -mfpv=neon -mfloat-abi=hard -I. -I/home/ol/1803_sec/ql-ol-extsdk-ec200acntar02a01m2gv01/ql-sysroots/usr/include
-I/home/ol/1803_sec/ql-ol-extsdk-ec200acntar02a01m2gv01/ql-sysroots/usr/include/ql-sdk -Wall -Wundef
/opt/ql_crosstools/ql-ec200a-1803e-gcc-8.4.0-v1-toolchain/bin/arm-openwrt-linux-gcc -o uart_test main.o -march=armv7-a
-marm -mfpv=neon -mfloat-abi=hard -L. -L/home/ol/1803_sec/ql-ol-extsdk-ec200acntar02a01m2gv01/ql-sysroots/usr -L/home
/ol/1803_sec/ql-ol-extsdk-ec200acntar02a01m2gv01/ql-sysroots/usr/lib -lql_sdk -lpthread -lql_sys_log -lql_lib_ipc -lql
lib_utils
ol@ol:~/1803_sec/ql-ol-extsdk-ec200acntar02a01m2gv01/sample/uart$ ls
main.c main.o Makefile uart_test
ol@ol:~/1803_sec/ql-ol-extsdk-ec200acntar02a01m2gv01/sample/uart$
```

4.2. 功能验证

参考第 4.1 章编译并生成可执行文件 *uart_test* 后，可参考如下章节所述步骤完成功能验证。

4.2.1. 未使能流控

下图为配置为无流控的部分代码截图：

```
/* Start: If need, to modify uart dcb config */
ST_UARTDCB dcb = {
    .flowctrl = FC_NONE, //none flow control
    .databit = DB_CS8, //databit: 8
    .stopbit = SB_1, //stopbit: 1
    .parity = PB_NONE, //parity check: none
    .baudrate = baudRate //baudrate: 115200
};
```

步骤一：依次执行以下两行命令将可执行文件 `uart_test` 上传至模块，并且修改执行权限：

```
adb push <uart_test 在上位机路径> <模块内部路径，如/tmp>
chmod 777 /tmp/uart_test
```

步骤二：以回环测试为例，将主串口的 TX 和 RX 连接。

步骤三：执行 `uart_test` 文件，验证 UART 功能是否成功（如下所示发送和接受一致，即表示成功）。

```
root@openwrt:/tmp# ./uart_test 115200
< OpenLinux: UART example >
SET DCB ret: 0
GET DCB ret: 0: baudrate: 115200, flowctrl: 0, databit: 8, stopbit: 1, paritybit: 0
< write(fd=3)=40
> read(uart)=40:uart test, +=_0)9(8*7&6^5%4$3#2@1!`~)
< write(fd=3)=40
> read(uart)=40:uart test, +=_0)9(8*7&6^5%4$3#2@1!`~)
< write(fd=3)=40
> read(uart)=40:uart test, +=_0)9(8*7&6^5%4$3#2@1!`~)
< write(fd=3)=40
> read(uart)=40:uart test, +=_0)9(8*7&6^5%4$3#2@1!`~)
```

接收到的数据

4.2.2. 使能硬件流控

步骤一：修改流控模式为硬件流控（即 `FC_RTSCCTS`）：

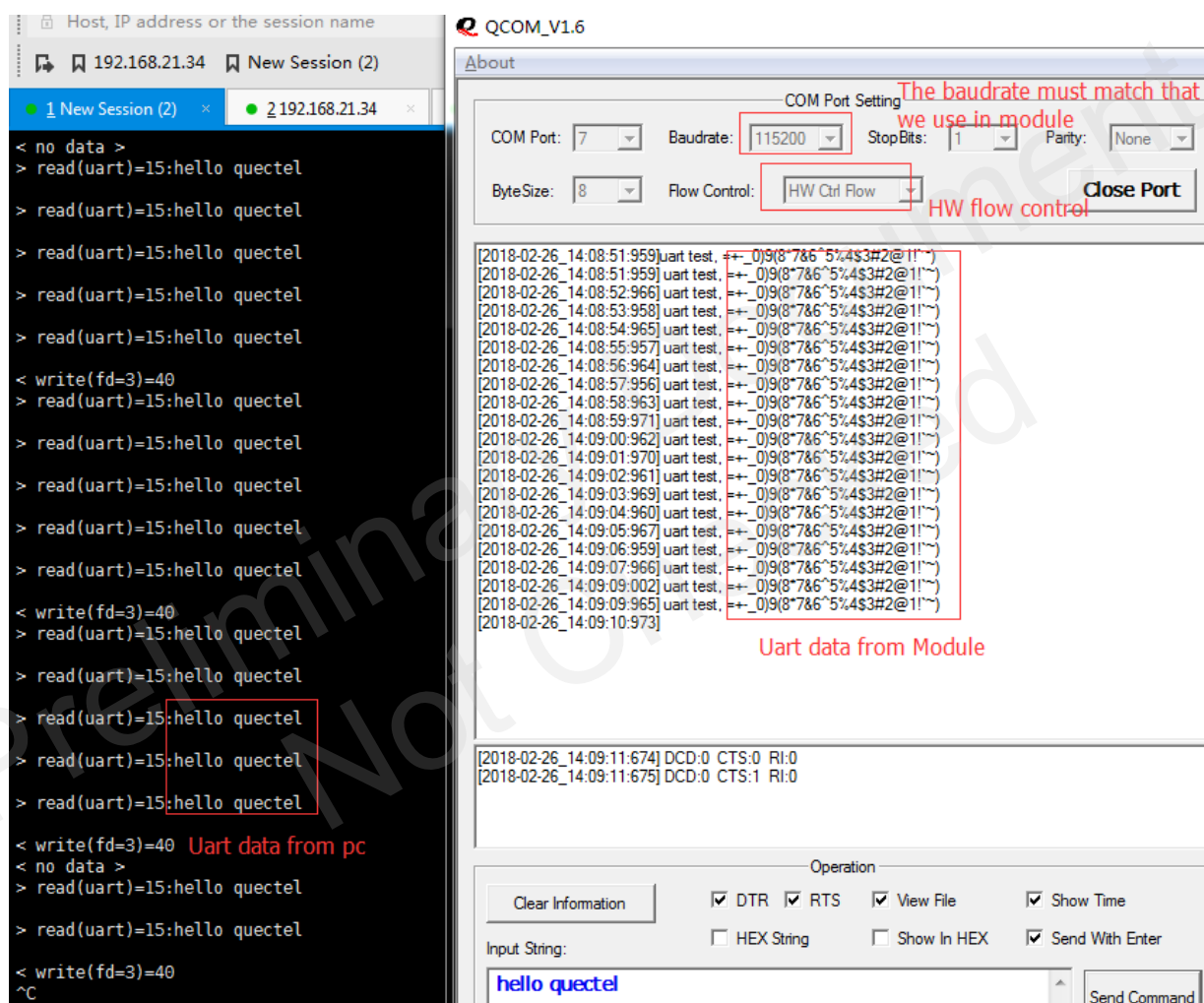
```
/* Start: If need, to modify uart dcb config */
ST_UARTDCB dcb = {
    .flowctrl = FC_RTSCCTS, //none flow control
    .databit = DB_CS8, //databit: 8
    .stopbit = SB_1, //stopbit: 1
    .parity = PB_NONE, //parity check: none
    .baudrate = baudRate //baudrate: 115200
};
```

步骤二：依次执行以下两行命令将可执行文件 `uart_test` 上传至模块，并且修改执行权限：

```
adb push <uart_test 在上位机路径> <模块内部路径，如/tmp>
chmod 777 /tmp/uart_test
```

步骤三：将 MAIN_UART 与 PC 连接。

步骤四：执行 `uart_test 115200`，并在上位机使用对应的波特率以及硬件流控方式打开对应 COM 端口。验证串口功能，使用 LTE OPEN EVB 和上位机通信以进行功能测试，如下所示：



5 附录 参考文档及术语缩写

表 5: 参考文档

文档名称
[1] Quectel_EC200A-CN(TA)_QuecOpen_快速开发指导
[2] Quectel_EC200A-CN(TA)_QuecOpen_GPIO_Configuration

表 6: 术语缩写

缩写	英文描述	中文描述
API	Application Programming Interface	应用程序编程接口
CPU	Central Processing Unit	中央处理器
EVB	Evaluation Board	评估板
GPIO	General-Purpose Input/Output	通用型输入/输出
I/O	Input/Output	输入/输出
IoT	Internet of Things	物联网
OD	Open Drain	开漏
SDK	Software Development Kit	软件开发工具包
UART	Inter-Integrated Circuit	集成电路总线